Java + Generative AI

The Complete Developer's Guide

From Zero to Production-Ready AI Applications

Spring Boot | LangChain4j | RAG | Agents | Ollama | Vector Databases

Includes: Deep Concepts • Full Code Examples • Interview Q&A; • Project Ideas

CHAPTER 0

How to Read This Book

This book is written for Java developers who want to build real AI-powered applications. It is structured so that each chapter builds on the previous one. Follow the order below for best results.

Recommended Reading Order

Chapter	Topic	Why Read It
Chapter 1	What is Generative AI?	Start here. Understand what LLMs are and how they w
Chapter 2	Core Concepts	Tokens, Embeddings, Context Window — must-know
Chapter 3	Prompt Engineering	Learn how to write good prompts before writing any co
Chapter 4	RAG — Retrieval Augmented Generation	The #1 most used Gen AI pattern in enterprise apps.
Chapter 5	Spring Boot + LangChain4j Setup	Set up your dev environment and write your first AI ap
Chapter 6	AI Agents and Tool Calling	Make AI that takes real actions, not just answers ques
Chapter 7	Vector Databases	Deep dive into how semantic search actually works.
Chapter 8	Ollama — Your Own Free Local LLM	Run AI on your own machine with zero cost.
Chapter 9	When to Use Gen AI	Know exactly when AI helps and when it hurts.
Chapter 10	Building Real Projects	Four complete project blueprints you can actually build
Chapter 11	Production Best Practices	Security, scaling, monitoring, cost control.
Chapter 12	Interview Preparation	50+ interview questions with detailed answers.

■ If you are completely new to AI, read Chapters 1–3 carefully before touching any code. If you already know AI basics, jump to Chapter 4.

■ *Every code example in this book is written in Java 17 with Spring Boot 3.2 and LangChain4j 0.31. The concepts apply to any version.*

CHAPTER 1

What is Generative AI?

Generative AI refers to artificial intelligence systems that can generate new content — text, images, code, audio, or video — based on patterns learned from massive amounts of training data. The most important type for Java developers is the Large Language Model (LLM).

1.1 What is a Large Language Model (LLM)?

A Large Language Model is a neural network trained on billions of words of text. It learns statistical patterns between words and can predict what word comes next in a sequence. This simple mechanism — predict the next word — gives rise to an astonishing ability to understand and generate human language.

Famous LLMs include OpenAI's GPT-4o, Google's Gemini, Anthropic's Claude, and open-source models like Meta's Llama 3 and Mistral. These models have hundreds of billions of parameters (numerical weights learned during training).

1.2 How Does an LLM Work? (Simple Explanation)

Think of it in three steps:

• INPUT: You send text (called a prompt) to the model.

• PROCESSING: The model runs your text through billions of mathematical operations.

• OUTPUT: The model generates a response, one token at a time.

// LLM API Call

```
// Calling an LLM is just an HTTP API call POST https://api.openai.com/v1/chat/completions { "model": "gpt-4o", "messages": [ { "role": "system", "content": "You are a helpful assistant." }, { "role": "user", "content": "What is Java?" } ] } // Response: { "choices": [{ "message": { "role": "assistant", "content": "Java is a high-level, object-oriented programming language..." } }] }
```

1.3 Types of AI Models You Will Use

Model Type	What It Does	Examples	When You Use It
Chat / Completion	Understands and generates text	GPT-4o, Llama3, Mistral	Chatbots, Q&A, summarization
Embedding	Converts text to numbers (vectors)	text-embedding-ada-002, nomic-embed	Semantic search, RAG
Image Generation	Creates images from text	DALL-E 3, Stable Diffusion	Product images, UI mockups
Code Generation	Writes and explains code	CodeLlama, GPT-4o	Code review, generation

1.4 What Can You Build with Java + Gen AI?

• Customer support chatbots that know your company's policies

• Document Q&A; systems (upload PDF, ask questions)

• Code review and generation tools

• Intelligent search engines (understand intent, not just keywords)

- Automated report and email generation
- Multi-language customer support
- Data analysis with natural language queries
- AI agents that take real actions in your systems

■ Gen AI does NOT replace traditional programming. It adds a layer of language understanding on top of your existing Java applications.

CHAPTER 2

Core Concepts You Must Know

2.1 Tokens — The Currency of LLMs

LLMs do not process text character by character or word by word. They process TOKENS. A token is roughly 4 characters or 0.75 words in English.

// Token Counting

```
// Token examples: "Hello" → 1 token "Hello, how are you?" → 5 tokens "Generative AI" → 3 tokens "Java Spring Boot" → 4 tokens // Rule of thumb: // 1,000 tokens ≈ 750 words ≈ 1.5 pages of text // 128,000 tokens ≈ a 100-page book
```

Tokens matter for three reasons:

• COST: OpenAI charges per token. More tokens = more money.

• CONTEXT WINDOW: Each model has a maximum token limit (GPT-4o = 128K tokens).

• SPEED: More tokens in the prompt = slower response.

2.2 Context Window

The context window is the maximum amount of text (measured in tokens) that an LLM can see at one time. This includes BOTH your input (prompt) AND the model's output (response).

Model	Context Window	Approx. Pages
GPT-3.5-turbo	16,000 tokens	~12 pages
GPT-4o	128,000 tokens	~96 pages
Claude 3.5 Sonnet	200,000 tokens	~150 pages
Llama 3 (8B) via Ollama	8,192 tokens	~6 pages
Mistral 7B via Ollama	32,768 tokens	~24 pages

■■ If your document is larger than the context window, you CANNOT just paste it into the prompt. This is exactly why RAG (Chapter 4) was invented.

2.3 Embeddings — Text as Numbers

An embedding is a way of representing text as a list of numbers (a vector). The key insight is that similar text produces similar vectors. This is the mathematical foundation of semantic search and RAG.

// Embedding Concept

```
"I love Java programming" → [0.23, -0.87, 0.45, 0.12, ...] (1536 numbers) "I enjoy coding in Java" → [0.22, -0.85, 0.44, 0.11, ...] (very similar!) "I love eating pizza" → [0.91, 0.34, -0.67, 0.88, ...] (very different) // Similar MEANING → Similar NUMBERS → Close in vector space // Different MEANING → Different NUMBERS → Far in vector space
```

2.4 Cosine Similarity

Cosine similarity measures how similar two vectors are. It calculates the angle between them. A score of 1.0 means identical meaning; 0.0 means completely unrelated.

// Cosine Similarity Scale

```
Cosine Similarity Score Interpretation: 1.0 → Identical meaning ("Java" vs "Java") 0.9+ → Very similar ("refund policy" vs "return policy") 0.7+ → Related ("order status" vs "where is my package?") 0.5 → Somewhat related 0.0 → Unrelated ("Java" vs "pizza") -1.0 → Opposite meaning
```

2.5 Temperature — Controlling Creativity

Temperature is a parameter (0.0 to 2.0) that controls how random or creative the LLM's responses are.

Temperature	Behavior	Best For
0.0	Deterministic, always same answer	Fact extraction, classification, SQL
0.3	Consistent, slightly varied	Q&A, RAG, customer support
0.7	Balanced (default)	General chat, summarization
1.0+	Creative, unpredictable	Story writing, brainstorming

■ For RAG and factual Q&A;, always use temperature 0.0 to 0.3 to minimize hallucination.

CHAPTER 3

Prompt Engineering

Prompt engineering is the art of writing inputs to LLMs that produce the best outputs. A good prompt can be the difference between a useless response and a perfect one. This is a critical skill for every Java Gen AI developer.

3.1 Types of Prompts

System Prompt

Sets the role, rules, and behavior of the AI. Sent once at the beginning.

// System Prompt Example

```
// System prompt — tells AI WHO it is and HOW to behave "You are a professional customer support agent for TechCorp. - Answer ONLY questions related to our products. - Always be polite and empathetic. - If you cannot help, say: 'Let me connect you to a human agent.' - Never reveal internal company information. - Always respond in the same language as the customer." // This shapes EVERY response the AI gives in this session.
```

User Prompt

The actual question or instruction from the user.

Assistant Prompt

Previous AI responses, included to give context in multi-turn conversations.

3.2 Prompting Techniques

Zero-Shot Prompting

Ask the model to do something without giving any examples.

// Zero-Shot

```
// Zero-shot: no examples, just ask directly "Classify this customer message as COMPLAINT, INQUIRY, or COMPLIMENT: Message: 'My order arrived broken and the packaging was terrible!'" // Model output: COMPLAINT
```

Few-Shot Prompting

Give 2-3 examples before asking the real question. Much more accurate.

// Few-Shot

```
// Few-shot: give examples first, then ask "Classify customer messages. Examples: Message: 'Your app is amazing!' → COMPLIMENT Message: 'Where is my order?' → INQUIRY Message: 'This is broken!' → COMPLAINT Now classify: Message: 'I have been waiting 2 weeks for my delivery.'" // Model output: COMPLAINT (accurate because of examples!)
```

Chain-of-Thought Prompting

Tell the model to think step by step. Dramatically improves reasoning.

// Chain-of-Thought

```
// Add "think step by step" for complex reasoning "A customer ordered 3 items at Rs.500 each. They used a 10% discount coupon and paid with a credit card that gives 5% cashback. How much
```

```
did they actually pay? Think step by step before giving the final answer." // Model thinks: // Step 1: Total = 3 x 500 = Rs.1500 // Step 2: After 10% discount = 1500 - 150 = Rs.1350 // Step 3: After 5% cashback = 1350 - 67.50 = Rs.1282.50 // Final Answer: Rs.1282.50
```

3.3 Prompt Templates in LangChain4j

// Prompt Templates

```
// Define a reusable prompt template with variables interface SupportAgent { @SystemMessage(""" You are a support agent for {{companyName}}. You help customers with orders and product queries. Always be professional and respond in {{language}}. """) @UserMessage("Customer {{customerName}} asks: {{question}}") String chat( @V("companyName") String company, @V("customerName") String name, @V("language") String lang, @V("question") String question ); } // Usage: String response = agent.chat("TechCorp", "Rahul", "Hindi", "refund kaise karen?"); // AI responds in Hindi!
```

3.4 Common Mistakes to Avoid

Mistake	Problem	Fix
Vague instructions	AI guesses what you want	Be specific: 'Reply in 2 sentences max'
No role definition	Generic unhelpful responses	Always add a system prompt
Asking multiple things	AI picks one, ignores rest	One question per prompt
No output format specified	Unstructured response	Say: 'Respond in JSON format'
High temperature for facts	Hallucinations	Use temperature 0.0-0.3 for facts

CHAPTER 4

RAG — Retrieval Augmented Generation

RAG is the most important Gen AI pattern for enterprise Java developers. It solves the fundamental problem that LLMs do not know your private data, company documents, or recent information. Understanding RAG deeply will make you stand out in any interview.

4.1 The Problem RAG Solves

Without RAG, LLMs have three critical limitations:

• Knowledge Cutoff: GPT-4o was trained on data up to a certain date. It knows nothing after that.

• No Private Data: It does not know your company policies, product catalog, or customer orders.

• Hallucination: When it does not know the answer, it may confidently make one up.

RAG fixes all three by giving the LLM real, relevant information at the time of the query.

4.2 RAG Architecture — The Two Phases

Phase 1: Indexing (Done Once / Offline)

```
// Indexing Phase
INDEXING PIPELINE: Your Documents (PDF, Word, DB, Website) | v [1] LOAD DOCUMENT | v [2] CHUNK (split into 500-token pieces with 50-token overlap) | v [3] EMBED each chunk → vector [0.23, -0.87, 0.45, ...] | v [4] STORE in Vector Database (Pinecone / pgvector / Weaviate) Result: Your documents are now searchable by MEANING, not keywords.
```

Phase 2: Querying (Every User Request)

```
// Query Phase
QUERY PIPELINE: User Question: "What is your refund policy?" | v [1] EMBED the question → vector [0.21, -0.85, 0.44, ...] | v [2] SEARCH Vector DB → find top-3 most similar chunks | v Retrieved chunks: Chunk 1: "Returns accepted within 30 days..." Chunk 2: "Damaged items refunded within 60 days..." Chunk 3: "Refunds processed in 5-7 business days..." | v [3] BUILD PROMPT: System: "Answer ONLY from the context below." Context: [Chunk 1] [Chunk 2] [Chunk 3] User: "What is your refund policy?" | v [4] LLM generates accurate, grounded answer ✓
```

4.3 Chunking Strategies

Chunking is splitting large documents into smaller pieces. Bad chunking = bad retrieval = wrong answers. Here are the main strategies:

Strategy	How It Works	Best For
Fixed Size	Split every N tokens regardless of content	Simple, fast, good baseline
Sentence-based	Split at sentence boundaries	Articles, blogs, paragraphs
Recursive	Try paragraph, then sentence, then word	Mixed content (recommended)
Sliding Window	Chunks overlap by N tokens	Prevents losing context at boundaries
Semantic	Group by topic/meaning	Complex, technical documents

■ Use recursive chunking with 500 tokens and 50-token overlap as your default. This works well for 90% of use cases.

4.4 Full RAG Code with LangChain4j

// Complete RAG Implementation

```
@Service public class RagService { private final EmbeddingModel embeddingModel; private final EmbeddingStore&lt;TextSegment&gt; embeddingStore; private final ChatLanguageModel llm; // STEP 1: INDEX a document (call once per document) public void indexDocument(String filePath) { Document doc = FileSystemDocumentLoader.loadDocument(filePath); // Chunk: 500 tokens, 50 overlap DocumentSplitter splitter = DocumentSplitters.recursive(500, 50); EmbeddingStoreIngestor.builder() .documentSplitter(splitter) .embeddingModel(embeddingModel) .embeddingStore(embeddingStore) .build() .ingest(doc); System.out.println("Document indexed!"); } // STEP 2: QUERY (call on every user question) public String ask(String question, String sessionId) { // Set up retriever: search vector DB EmbeddingStoreContentRetriever retriever = EmbeddingStoreContentRetriever.builder() .embeddingStore(embeddingStore) .embeddingModel(embeddingModel) .maxResults(3) // top 3 chunks .minScore(0.7) // only relevant chunks .build(); // Build RAG-powered agent RagAgent agent = AiServices.builder(RagAgent.class) .chatLanguageModel(llm) .contentRetriever(retriever) // RAG enabled! .chatMemory(MessageWindowChatMemory.withMaxMessages(10)) .build(); return agent.chat(question); } } // Agent interface interface RagAgent { @SystemMessage(""" Answer ONLY from the provided context. If not found, say: 'I could not find that in our documents.' """) String chat(String question); }
```

4.5 RAG vs Fine-Tuning — When to Use Which

Aspect	RAG	Fine-Tuning
What it does	Injects knowledge at runtime	Bakes knowledge into model weights
Cost	Low (vector DB + API calls)	Very high ($500 to $100,000+)
Update data	Instant (re-index)	Retrain model (days/weeks)
Best for	Dynamic, private, recent data	Changing behavior/style/tone
Hallucination risk	Low (grounded in context)	Medium (model may still hallucinate)
Java complexity	Moderate	Requires Python/ML knowledge

■ *Rule of thumb: Use RAG for KNOWLEDGE problems. Use Fine-tuning for BEHAVIOR problems. Most enterprise apps only ever need RAG.*

CHAPTER 5

Spring Boot + LangChain4j Setup

This chapter walks you through setting up your first Spring Boot Gen AI project from scratch. Every line of code is explained.

5.1 Prerequisites

• Java 17 or higher installed

• Maven 3.8+ installed

• An OpenAI API key (get from platform.openai.com) OR Ollama installed (Chapter 8)

• Your favorite IDE (IntelliJ IDEA recommended)

5.2 Maven Dependencies

// pom.xml

```
&lt;!-- pom.xml - Essential dependencies --&gt; &lt;!-- LangChain4j Core --&gt;
&lt;dependency&gt; &lt;groupId&gt;dev.langchain4j&lt;/groupId&gt;
&lt;artifactId&gt;langchain4j&lt;/artifactId&gt; &lt;version&gt;0.31.0&lt;/version&gt;
&lt;/dependency&gt; &lt;!-- LangChain4j Spring Boot Starter --&gt; &lt;dependency&gt;
&lt;groupId&gt;dev.langchain4j&lt;/groupId&gt;
&lt;artifactId&gt;langchain4j-spring-boot-starter&lt;/artifactId&gt;
&lt;version&gt;0.31.0&lt;/version&gt; &lt;/dependency&gt; &lt;!-- OpenAI Integration --&gt;
&lt;dependency&gt; &lt;groupId&gt;dev.langchain4j&lt;/groupId&gt;
&lt;artifactId&gt;langchain4j-open-ai-spring-boot-starter&lt;/artifactId&gt;
&lt;version&gt;0.31.0&lt;/version&gt; &lt;/dependency&gt; &lt;!-- OR for Ollama (local free
LLM) --&gt; &lt;dependency&gt; &lt;groupId&gt;dev.langchain4j&lt;/groupId&gt;
&lt;artifactId&gt;langchain4j-ollama&lt;/artifactId&gt; &lt;version&gt;0.31.0&lt;/version&gt;
&lt;/dependency&gt;
```

5.3 application.yml Configuration

// application.yml

```
# application.yml langchain4j: open-ai: chat-model: api-key: ${OPENAI_API_KEY} # set as env
variable, never hardcode! model-name: gpt-4o temperature: 0.7 max-tokens: 1000
embedding-model: api-key: ${OPENAI_API_KEY} model-name: text-embedding-ada-002 # For Ollama
(no API key needed): langchain4j: ollama: chat-model: base-url: http://localhost:11434
model-name: llama3 temperature: 0.7
```

5.4 Your First AI Endpoint — Step by Step

// First AI Endpoint

```
// Step 1: Define the AI interface interface MyAI { @SystemMessage("You are a helpful Java
expert.") String chat(String message); } // Step 2: Create the service @Service public class
MyAIService { private final MyAI ai; public MyAIService(ChatLanguageModel llm) { this.ai =
AiServices.builder(MyAI.class) .chatLanguageModel(llm)
.chatMemory(MessageWindowChatMemory.withMaxMessages(10)) .build(); } public String ask(String
question) { return ai.chat(question); } } // Step 3: Create the REST controller
@RestController @RequestMapping("/api") public class AIController { @Autowired private
MyAIService aiService; @PostMapping("/chat") public
```

```
ResponseEntity&lt;Map&lt;String,String&gt;&gt; chat( @RequestBody Map&lt;String,String&gt;
req) { String answer = aiService.ask(req.get("message")); return
ResponseEntity.ok(Map.of("response", answer)); } } // Test with curl: // curl -X POST
http://localhost:8080/api/chat // -H "Content-Type: application/json" // -d '{"message":"What
is Spring Boot?"}'
```

CHAPTER 6

AI Agents and Tool Calling

A regular LLM answers questions from its training. An AI agent can TAKE ACTIONS — query your database, call external APIs, send emails, calculate values, and more. This is what makes Gen AI truly powerful for enterprise applications.

6.1 What is an AI Agent?

// Agent vs LLM

```
REGULAR LLM: User → LLM → Answer (can only use training data, may hallucinate) AI AGENT: User → LLM → "I need real data" → Call Tool → Get Result → LLM → Answer (uses REAL data from YOUR systems, no hallucination on facts) Example: User: "What is the status of order ORD-12345?" Without agent: LLM guesses or says "I don't know" ✗ With agent: LLM thinks: "I need to look up this order" LLM calls: getOrderById("ORD-12345") DB returns: {status: "SHIPPED", expectedDelivery: "Jan 25"} LLM responds: "Your order ORD-12345 is shipped, arriving Jan 25." ✓
```

6.2 The ReAct Pattern

ReAct (Reason + Act) is the most popular agent pattern. The LLM alternates between thinking (reasoning) and acting (calling tools) until it has a complete answer.

// ReAct Loop

```
User: "What is the weather in Mumbai and how does it compare to Delhi?" [THOUGHT] I need to get weather data for both cities. [ACTION] call getWeather("Mumbai") [RESULT] "Mumbai: 32 deg C, humidity 85%" [THOUGHT] Now I need Delhi's weather. [ACTION] call getWeather("Delhi") [RESULT] "Delhi: 38 deg C, humidity 45%" [THOUGHT] I have both. I can now compare and answer. [ANSWER] "Mumbai is 32 deg C with high humidity. Delhi is hotter at 38 deg C but less humid. Delhi is 6 degrees warmer today." ✓
```

6.3 Building Tools in Java

// Tool Definition

```
// Tools are just regular Java methods with @Tool annotation // The description tells the LLM WHEN to call this tool @Component public class OrderTool { @Autowired private OrderRepository orderRepo; // LLM reads this description to decide when to call this method @Tool("Get order details by order ID. " + "Call this when the customer provides an order number.") public String getOrderById(String orderId) { return orderRepo.findById(orderId) .map(o -> "Order: " + o.getId() + ", Status: " + o.getStatus() + ", Total: Rs." + o.getTotal()) .orElse("Order not found: " + orderId); } @Tool("Get all orders for a customer. " + "Call this when customer provides their email address.") public String getOrdersByEmail(String email) { List&lt;Order&gt; orders = orderRepo.findByEmail(email); if (orders.isEmpty()) return "No orders found for: " + email; return orders.stream() .map(Order::toString) .collect(Collectors.joining(", ")); } @Tool("Calculate refund amount for an order.") public String calculateRefund(String orderId, double percentage) { return orderRepo.findById(orderId) .map(o -> "Refund: Rs." + (o.getTotal() * percentage / 100)) .orElse("Order not found"); } }
```

6.4 Wiring Agent with Tools

// Agent with Tools + RAG

```
@Service public class AgentService { private final CustomerSupportAgent agent; public
AgentService(ChatLanguageModel llm, EmbeddingModel embModel, EmbeddingStore&lt;TextSegment&gt;
store, OrderTool orderTool) { EmbeddingStoreContentRetriever retriever =
EmbeddingStoreContentRetriever.builder() .embeddingStore(store) .embeddingModel(embModel)
.maxResults(3).minScore(0.6) .build(); this.agent =
AiServices.builder(CustomerSupportAgent.class) .chatLanguageModel(llm)
.contentRetriever(retriever) // RAG for documents .tools(orderTool) // Tools for real data
.chatMemory( MessageWindowChatMemory.withMaxMessages(20)) .build(); } public String
chat(String message) { return agent.chat(message); } } interface CustomerSupportAgent {
@SystemMessage(""" You are a professional customer support agent. Use your tools to look up
order information. Use your knowledge base for policy questions. Always be helpful and
accurate. """) String chat(String message); }
```

CHAPTER 7

Vector Databases

A vector database is a specialized database for storing and searching embeddings (vectors). It is the backbone of RAG. Without a vector database, you cannot do semantic search at scale.

7.1 Why Not Use a Regular Database?

// SQL vs Vector DB

```
// Regular SQL - exact keyword match only: SELECT * FROM docs WHERE content LIKE '%refund%' // Misses: "money back", "return policy", "reimbursement" // Vector DB - semantic search: "What is your money-back guarantee?" // Finds: refund policy, return terms, reimbursement policy // → Understands MEANING, not just keywords ✓
```

7.2 Vector Database Comparison

Database	Type	Best For	Free Tier	Java Support
Pinecone	Cloud	Production, scalable	Yes (1 index)	Via LangChain4j
pgvector	PostgreSQL ext.	Already using Postgres	Yes (self-host)	Via LangChain4j
Weaviate	Open source	Hybrid search	Yes (self-host)	Via LangChain4j
Qdrant	Open source	High performance	Yes (self-host)	Via LangChain4j
Chroma	Open source	Local dev/testing	Yes	Via LangChain4j
In-Memory	In-process	Dev, testing, demos	Yes (built-in)	LangChain4j built-in

7.3 Using pgvector (PostgreSQL)

If your app already uses PostgreSQL, pgvector is the easiest choice. No new infrastructure needed.

// pgvector SQL

```
-- 1. Enable the vector extension CREATE EXTENSION IF NOT EXISTS vector; -- 2. Create table with vector column CREATE TABLE document_chunks ( id SERIAL PRIMARY KEY, content TEXT NOT NULL, source VARCHAR(255), embedding vector(1536) -- 1536 for OpenAI embeddings ); -- 3. Create index for fast similarity search CREATE INDEX ON document_chunks USING ivfflat (embedding vector_cosine_ops); -- 4. Similarity search query SELECT content, 1 - (embedding &lt;=&gt; '[0.1,0.2,...]') AS score FROM document_chunks ORDER BY embedding &lt;=&gt; '[0.1,0.2,...]' LIMIT 5;
```

// pgvector Java Config

```
// Java: Use PgVectorEmbeddingStore EmbeddingStore&lt;TextSegment&gt; store = PgVectorEmbeddingStore.builder() .host("localhost") .port(5432) .database("mydb") .user("postgres") .password("password") .table("document_chunks") .dimension(1536) .build();
```

7.4 Using Pinecone (Cloud)

// Pinecone Java

```
// 1. Create index at pinecone.io (dimension=1536, metric=cosine) // 2. Get API key from Pinecone dashboard EmbeddingStore&lt;TextSegment&gt; store = PineconeEmbeddingStore.builder() .apiKey(System.getenv("PINECONE_API_KEY")) .index("my-knowledge-base") .nameSpace("company-docs") // isolate by namespace (multi-tenant!) .build(); // That's it! Same API as InMemoryEmbeddingStore. // Production-ready, scales to billions of vectors.
```

CHAPTER 8

Ollama — Your Own Free Local LLM

Ollama lets you run powerful open-source LLMs on your own machine — completely free, completely private, no internet required. This is the best option for development, learning, and privacy-sensitive applications.

8.1 Install Ollama (5 minutes)

// Install Ollama

```
# Linux / Mac: curl -fsSL https://ollama.com/install.sh | sh # Windows: # Download installer from https://ollama.com/download # Verify installation: ollama --version
```

8.2 Download and Run Models

// Ollama Models

```
# Download and run Llama 3 (8B parameters, ~5GB) ollama pull llama3 ollama run llama3 # Other popular models: ollama pull mistral # Fast, good for chat ollama pull codellama # Specialized for code ollama pull phi3 # Small but powerful (3.8B) ollama pull gemma # Google's open model ollama pull nomic-embed-text # Embedding model for RAG # List downloaded models: ollama list # Ollama runs as a server at: http://localhost:11434
```

8.3 Connect Spring Boot to Ollama

// Ollama Config

```
// application.yml - use Ollama instead of OpenAI langchain4j: ollama: chat-model: base-url: http://localhost:11434 # Ollama server model-name: llama3 # Model you downloaded temperature: 0.7 timeout: 120s # Local models are slower embedding-model: base-url: http://localhost:11434 model-name: nomic-embed-text # Free local embedding model
```

// Ollama Java Code

```
// Java code - exactly same as OpenAI! Just different config. @Service public class OllamaService { @Autowired private ChatLanguageModel chatModel; // auto-injected by LangChain4j @Autowired private EmbeddingModel embeddingModel; // nomic-embed-text public String ask(String question) { return chatModel.generate(question); // calls Ollama locally } } // ZERO API KEY NEEDED. ZERO COST. WORKS OFFLINE.
```

8.4 Model Comparison for Local Use

Model	Size	Speed	Quality	Best For
Phi-3 Mini	2.3GB	Very Fast	Good	Learning, low-end hardware
Llama 3 (8B)	4.7GB	Fast	Very Good	General chat, RAG
Mistral (7B)	4.1GB	Fast	Very Good	Chat, code
CodeLlama (13B)	7.4GB	Medium	Excellent	Code generation
Llama 3 (70B)	40GB	Slow	Near GPT-4	High-end server only

■ Start with Llama 3 (8B) for development. It runs on most laptops with 8GB RAM and is good enough for most tasks.

■■ Local models are slower than cloud APIs. Llama 3 (8B) takes 3-10 seconds per response on a typical laptop. For production, use cloud APIs.

CHAPTER 9

When to Use Gen AI (and When Not To)

This is the most important strategic chapter. Using AI where it does not belong is expensive, slow, and unreliable. Using it where it shines creates incredible value. Know the difference.

9.1 USE Gen AI When...

Use Case	When This Applies	Example
Natural Language Understanding	Users type free-form questions in any language	Customer support chatbot
Unstructured Data Processing	Extracting info from PDFs, emails, documents	Legal document analysis
Content Generation	Writing summaries, emails, descriptions at scale	Product description generator
Semantic Search	Find relevant content by meaning, not keywords	Knowledge base search
Code Understanding	Explain, review, or generate code	Internal developer tool
Multi-language Support	Respond in user's language automatically	Global customer support
Complex Reasoning	Multi-step problems combining multiple data sources	Financial advice assistant

9.2 DO NOT Use Gen AI When...

• PRECISE CALCULATIONS: Use a calculator, not an LLM. LLMs can make arithmetic errors.

• REAL-TIME DATA: LLMs have no live data. Use APIs + Agent tools for that.

• SIMPLE CRUD: Creating, reading, updating, deleting records. Just use JPA/SQL.

• GUARANTEED ACCURACY: Medical dosing, financial transactions, legal contracts — too high stakes for LLMs alone.

• HIGH VOLUME SIMPLE TASKS: If a simple regex or SQL query does it, don't pay per LLM token.

• SECURITY-CRITICAL DECISIONS: Never let an LLM decide who gets access to what.

9.3 Decision Framework

// Decision Framework

```
Ask these questions before adding Gen AI: 1. Does this involve UNDERSTANDING or GENERATING natural language? YES → Consider Gen AI NO → Probably don't need it 2. Is the data UNSTRUCTURED (text, PDFs, emails)? YES → Gen AI is very helpful NO → Traditional DB queries may be better 3. Is 95% accuracy good enough, or do you need 100%? 95% OK → Gen AI can work (with RAG to improve accuracy) 100% needed → Do NOT rely on LLM alone 4. Is the response latency acceptable? (1-5 seconds for LLM) YES → Gen AI is fine NO → LLMs may be too slow (need caching strategy) 5. Is the per-query cost acceptable? (~$0.001-0.01 per query) YES → Use cloud LLM NO → Consider local LLM (Ollama) or batch processing
```

9.4 Real Industry Examples

Company / Domain	What They Use Gen AI For	Result
E-commerce (Flipkart-style)	Customer support + order lookup agent	60% fewer human support tickets
Banking	Document analysis for loan applications	10x faster processing
Healthcare	Medical report summarization	Doctors save 2 hrs/day

Company / Domain	What They Use Gen AI For	Result
Legal	Contract review and clause extraction	80% faster contract review
HR Tech	Resume screening and matching	Screens 1000 resumes in minutes
EdTech	Personalized tutoring chatbot	Students get instant help 24/7

CHAPTER 10

Building Real Projects

The best way to learn is to build. Here are four complete project blueprints that you can actually build and put on your resume. Each one demonstrates a different combination of Gen AI features.

Project 1: PDF Q&A; Chatbot

User uploads a PDF and asks questions about it. Classic RAG application.

Feature	Technology
PDF upload endpoint	Spring Boot Multipart
PDF text extraction	Apache PDFBox
Chunking	LangChain4j DocumentSplitters.recursive()
Embedding	OpenAI text-embedding-ada-002 or nomic-embed-text
Vector storage	InMemoryEmbeddingStore (dev), pgvector (prod)
Chat interface	LangChain4j AiServices + RAG
Memory	MessageWindowChatMemory

// Project 1 Blueprint

```
// Key endpoints: POST /api/upload - upload PDF, returns "indexed" POST /api/chat - send question, get AI answer from PDF // What you learn: // - Full RAG pipeline end to end // - File upload handling // - Session management // - How to structure a Gen AI Spring Boot project // Estimated build time: 2-3 days
```

Project 2: Customer Support Agent

AI chatbot for an e-commerce site. Answers policy questions + checks real order data.

Feature	Technology
Policy Q&A (RAG)	LangChain4j + InMemory/pgvector
Order lookup tool	@Tool annotation + JPA repository
Multi-turn chat	MessageWindowChatMemory per session
Hindi/English support	Multilingual system prompt
REST API	Spring Boot @RestController
Data	H2/PostgreSQL with sample orders

// Project 2 Blueprint

```
// What you learn: // - AI Agents with tool calling // - Combining RAG + Tools in one agent // - Per-user session memory // - Multi-language AI apps // Estimated build time: 3-5 days
```

Project 3: AI Code Reviewer

Paste Java code, get detailed review with bugs, performance, and security issues.

// Project 3 Blueprint

```
// Key component: interface CodeReviewAgent { @SystemMessage(""" You are a senior Java developer. Review code for: 1. Bugs and null pointer risks 2. Performance issues (N+1 queries,
```

```
memory leaks) 3. Security vulnerabilities (SQL injection, XSS) 4. Spring Boot best practices 5. SOLID principle violations Be specific. Give line numbers. Suggest fixes. """) @UserMessage("Review this code:\n```java\n{{code}}\n```") String review(@V("code") String code); } // Endpoint: POST /api/review { "code": "...java code..." } // Estimated build time: 1-2 days
```

Project 4: Semantic Job Search Engine

Search job listings by meaning. 'Looking for backend role' finds Java, Spring, microservices jobs.

// Project 4 Blueprint

```
// Architecture: // 1. Index all job descriptions as embeddings // 2. User searches in natural language // 3. Find semantically similar jobs // 4. Rank by relevance score // Key advantage over keyword search: // Query: "I want to work on distributed systems" // Finds: "Kafka engineer", "microservices developer", // "backend engineer (high scale)" // Even though none contain "distributed systems" literally! // Estimated build time: 2-3 days
```

CHAPTER 11

Production Best Practices

11.1 Security

• NEVER hardcode API keys. Always use environment variables or a secrets manager (AWS Secrets Manager, HashiCorp Vault).

• Validate all user input before sending to LLM — prevent prompt injection attacks.

• Scan LLM outputs for harmful content before sending to users.

• Rate-limit per user/IP using Bucket4j to prevent abuse and cost explosion.

• Log all LLM interactions for security audit and debugging.

• Use separate API keys per environment (dev, staging, prod).

11.2 Cost Control

Strategy	How It Helps	Savings
Response caching (Redis)	Same question → cached answer, no LLM call	30-50% cost reduction
Smaller model for simple tasks	Use GPT-3.5 for classification, GPT-4 for complex only	80% cheaper
Token optimization	Keep prompts concise, limit max_tokens	20-40% cheaper
Batch processing	Process non-urgent tasks in batches	50% cheaper (batch API)
Local LLM (Ollama)	Zero API cost for dev/non-critical tasks	100% savings

11.3 Error Handling and Resilience

// Error Handling

```
// Always handle LLM failures gracefully @Service public class ResilientAIService {
@Retryable( value = Exception.class, maxAttempts = 3, backoff = @Backoff(delay = 1000,
multiplier = 2) ) public String ask(String question) { return aiAgent.chat(question); }
@Recover public String recover(Exception e, String question) { log.error("LLM failed after 3
retries: {}", e.getMessage()); return "I am temporarily unavailable. Please try again in a
moment."; } }
```

11.4 Performance Optimization

• Cache frequent queries in Redis with a TTL of 1-24 hours depending on how dynamic your data is.

• Use async processing for non-real-time tasks (Kafka + background workers).

• Stream LLM responses (SSE) so users see text appearing immediately instead of waiting.

• Pre-warm your embedding store on startup by indexing documents before first user request.

• Use connection pooling for vector DB connections.

• Monitor p50, p95, p99 latencies with Prometheus + Grafana.

11.5 Production Architecture

// Production Architecture

```
Production Setup: User | v API Gateway (rate limiting, auth) | v Load Balancer | v Spring Boot App Pods (horizontal scaling) | | | v v v Redis OpenAI pgvector (cache, (or Ollama (embeddings) memory) cluster) | v PostgreSQL (orders, users, audit logs) | v Kafka (async ingestion pipeline) | v Worker Pods (PDF processing, embedding)
```

CHAPTER 12

Interview Preparation — 50 Q&A;

These are the most commonly asked questions in Java + Gen AI interviews, grouped by topic. Study them in order. The answers here are written as you should say them in an interview — clear, confident, and to the point.

Conceptual Questions

Q: What is RAG and why is it used?

A: RAG stands for Retrieval Augmented Generation. It solves the problem that LLMs don't know private or recent data. We index documents as vectors in a vector DB. At query time, we embed the user's question, find the most similar document chunks, and include them in the LLM prompt as context. The LLM then answers based on that real context instead of guessing.

Q: What is an embedding?

A: An embedding is a numerical vector representation of text. An embedding model converts text into a list of numbers (typically 1536 numbers for OpenAI). Similar text produces similar vectors. This allows us to do mathematical similarity search — finding chunks of text that are semantically related to a user's question.

Q: What is cosine similarity?

A: Cosine similarity measures the angle between two vectors. A score of 1.0 means identical meaning, 0.7+ means very similar, and 0.0 means unrelated. Vector databases use cosine similarity to find the most relevant document chunks for a given query.

Q: What is chunking and why does overlap matter?

A: Chunking splits large documents into smaller pieces that fit within the LLM context window. Overlap (typically 50 tokens) between chunks prevents losing important context that might fall at a chunk boundary. Without overlap, a sentence that spans two chunks might be split in a way that loses its meaning.

Q: What is hallucination and how do you prevent it?

A: Hallucination is when an LLM generates confident but false information not grounded in facts. Prevention: use RAG to provide real context, set a minimum similarity score so only relevant chunks are used, instruct the LLM to say 'I don't know' if the answer is not in the context, and include source citations so users can verify.

Q: What is a context window?

A: The context window is the maximum number of tokens an LLM can process in a single request — including both the input prompt and the output. GPT-4o has a 128K token context window. This is why we use chunking and RAG instead of pasting entire documents into the prompt.

Q: What is the difference between RAG and fine-tuning?

A: RAG retrieves external knowledge at runtime — it's cheap, updatable without retraining, and good for dynamic private data. Fine-tuning bakes knowledge into model weights during training — it's expensive, static, and best for changing the model's behavior or style. Most enterprise applications only need RAG.

Q: What is function/tool calling?

A: Tool calling allows the LLM to call predefined Java methods to get real data. The LLM reads the @Tool description and decides when to call each method. For example, when a user asks about an order, the LLM calls getOrderById(), gets the real DB result, and uses that to form the answer. This eliminates hallucination for structured data.

Q: What is the ReAct agent pattern?

A: ReAct stands for Reason and Act. The agent alternates between thinking about what it needs (Reason) and calling a tool to get it (Act), repeating until it has enough information to answer the user's question. It is the

most popular pattern for building AI agents.

Q: What is a vector database?

A: A vector database is specialized for storing and searching high-dimensional vectors (embeddings). Unlike SQL that matches exact values, a vector DB finds vectors that are mathematically close — enabling semantic similarity search. Examples: Pinecone, pgvector, Weaviate, Qdrant.

Technical Implementation Questions

Q: How do you implement streaming responses?

A: Use StreamingChatLanguageModel instead of ChatLanguageModel. The response comes back as a Flux<String> using Project Reactor. On the controller side, return Flux<String> with MediaType.TEXT_EVENT_STREAM_VALUE so the client receives tokens as they are generated, just like ChatGPT.

Q: How do you manage per-user conversation memory?

A: Use a ConcurrentHashMap<String, ChatMemory> keyed by sessionId. Each user gets their own MessageWindowChatMemory. In production, replace this with Redis and set a TTL (e.g., 1 hour) so sessions are automatically cleaned up.

Q: How do you handle OpenAI rate limits?

A: Use Spring's @Retryable annotation with exponential backoff — retry up to 3 times, doubling the wait each time (1s, 2s, 4s). Add a @Recover method that returns a fallback message if all retries fail. For bulk tasks, use the OpenAI Batch API which is 50% cheaper and not rate-limited.

Q: How do you secure an LLM application?

A: Store API keys in environment variables, never in code. Rate-limit per user using Bucket4j. Validate and sanitize user input to prevent prompt injection. Scan LLM outputs before sending to users. Log all LLM calls for audit. Use separate API keys per environment.

Q: How do you evaluate RAG quality?

A: Use four metrics: Faithfulness (is the answer grounded in the retrieved context?), Answer Relevance (does the answer actually address the question?), Context Recall (were the right chunks retrieved?), and Context Precision (were all retrieved chunks relevant?). The RAGAs framework automates these measurements.

Q: What is minScore and why is it important?

A: minScore is a threshold on the cosine similarity score. Only chunks with a score above this threshold are included as context. This prevents irrelevant chunks from being injected into the prompt, which would confuse the LLM. A good starting value is 0.6-0.7. Without minScore, even very irrelevant chunks get sent to the LLM.

Q: How do you implement multi-tenant RAG?

A: Use namespaces in your vector DB — one namespace per tenant. During indexing, store chunks in the tenant's namespace. During retrieval, filter by tenantId so each tenant only sees their own data. Pinecone and Weaviate both support namespaces natively.

Q: How do you handle documents larger than the context window?

A: This is exactly what RAG solves. Index the entire document in chunks. At query time, only retrieve the 3-5 most relevant chunks. The LLM never sees the whole document — only the parts relevant to the current question. For very large documents, also use parent-child chunking for better precision.

System Design Questions

Q: Design a RAG system for 10 million documents.

A: Use Kafka for async ingestion: documents flow through a queue to worker pods that chunk, embed, and store in Pinecone. For querying: API Gateway → Auth → Rate Limit → Query Service (embed query →

Pinecone search → rerank → LLM call) → Redis cache for frequent queries. Scale horizontally with Kubernetes.

Q: How would you reduce LLM costs in production?

A: Five strategies: 1) Cache responses in Redis — same questions get cached answers with no LLM call (30-50% savings). 2) Route simple tasks to cheaper models (GPT-3.5 for classification, GPT-4 for complex reasoning). 3) Optimize prompts — shorter prompts cost less. 4) Use batch API for non-urgent processing (50% cheaper). 5) Use local Ollama for development and non-critical features.

Q: How do you ensure LLM output quality in production?

A: Use LLM-as-judge pattern: after generating a response, a second LLM call evaluates it for accuracy, relevance, and hallucination. Log all responses. Collect user feedback (thumbs up/down). A/B test different prompts. Monitor RAGAs metrics continuously. Set up alerts when quality scores drop below threshold.

Quick-Fire Answers Cheat Sheet

Question	Answer
What does RAG stand for?	Retrieval Augmented Generation
What is a token?	~4 characters or 0.75 words
GPT-4o context window?	128,000 tokens (~96 pages)
Default embedding dimension?	1536 (OpenAI ada-002)
cosine similarity range?	-1.0 to 1.0 (1.0 = identical)
Best chunk size?	500 tokens with 50 token overlap
Temperature for facts?	0.0 to 0.3
LangChain4j tool annotation?	@Tool with descriptive text
RAG or fine-tune for new data?	RAG — cheaper and updatable
Free local LLM option?	Ollama with Llama3 or Mistral
Vector DB for PostgreSQL users?	pgvector extension
Best agent pattern?	ReAct (Reason + Act)
Per-user memory storage?	Map<sessionId, ChatMemory> or Redis
Prevent hallucination how?	RAG + minScore + 'say I dont know' instruction
LangChain4j or Spring AI?	LangChain4j has more features; Spring AI is simpler

APPENDIX

Glossary, Resources & Next Steps

Glossary — Every Term Explained

Agent: An LLM combined with tools, memory, and a loop that lets it take real actions.

Chunking: Splitting documents into smaller pieces (typically 500 tokens) for embedding.

Context Window: Maximum tokens an LLM can process in one request (input + output).

Cosine Similarity: Mathematical measure of angle between two vectors. 1.0 = identical, 0.0 = unrelated.

Embedding: A vector (list of numbers) representation of text. Similar text = similar vectors.

Fine-tuning: Training a pre-existing model on custom data to change its behavior or style.

Hallucination: LLM generating confident but false information.

LangChain4j: Java framework for building AI applications with agents, RAG, and memory.

LLM: Large Language Model. AI trained on massive text data to understand and generate language.

Namespace: Isolation unit in vector DBs. Used for multi-tenant RAG.

Ollama: Tool for running open-source LLMs locally, for free.

Prompt: The text input you send to an LLM.

RAG: Retrieval Augmented Generation. Fetching relevant document chunks and injecting into prompt.

ReAct: Reason + Act. Agent pattern where LLM alternates between thinking and calling tools.

System Prompt: Instructions that define the AI's role and behavior for an entire conversation.

Temperature: 0.0-2.0 parameter controlling randomness. 0.0 = deterministic, 1.0+ = creative.

Token: Basic unit of LLM text processing. ~4 characters or 0.75 words.

Tool Calling: LLM ability to call predefined Java methods to get real data.

Vector Database: Database optimized for storing and searching embeddings by similarity.

Essential Resources

Resource	URL / Location	What to Learn
LangChain4j Docs	docs.langchain4j.dev	Complete API reference
Spring AI Docs	docs.spring.io/spring-ai	Spring-native AI integration
OpenAI Docs	platform.openai.com/docs	API reference, models, pricing
Ollama	ollama.com	Download, model library
DeepLearning.AI	deeplearning.ai	Free short courses on RAG, agents
LangChain4j GitHub	github.com/langchain4j/langchain4j	Examples and source code
Pinecone Docs	docs.pinecone.io	Cloud vector DB setup
RAGAs	github.com/explodinggradients/ragas	RAG evaluation framework

Your 10-Week Learning Plan

Week	Goal	Action
Week 1	Understand fundamentals	Read Chapters 1-3 of this book
Week 2	RAG mastery	Read Chapter 4 + implement basic RAG
Week 3	Spring Boot setup	Chapter 5 + run first AI endpoint
Week 4	Agents & Tools	Chapter 6 + build customer support agent
Week 5	Vector DBs	Chapter 7 + connect pgvector or Pinecone
Week 6	Free local LLM	Chapter 8 + set up Ollama + Llama3
Week 7	Build Project 1	PDF Q&A chatbot (Chapter 10, Project 1)
Week 8	Build Project 2	Customer support agent (Chapter 10, Project 2)
Week 9	Production	Chapter 11 + add security, caching, monitoring
Week 10	Interview prep	Chapter 12 + practice all 50 Q&As out loud

---

You now have everything you need to become a Java + Gen AI developer. The concepts are deep but the path is clear. Build something. Break it. Fix it. Repeat. That is how every great engineer learned.

Good luck. You've got this. ■